VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



ASSIGNMENT 1 REPORT

DEEP LEARNING AND ITS APPLICATIONS

Course Code: CO3133 Class: CC01

SEMESTER 261

Group Name: Group 09 - Ryzen9

Instructor(s): Le Thanh Sach

Student(s): Hoang Kim Cuong - 2352145

Phan Tan Loc - 2352712

Tran Manh Thang - 2353124

HO CHI MINH CITY, OCTOBER 2026

Workload

No.	Full name	Student ID	Workload (%)	Assigned Report Section
1	Hoang Kim Cuong	2352145	100%	Problem and Data Description
2	Phan Tan Loc	2352712	100%	Methodology
3	Tran Manh Thang	2353124	100%	Implementation Results

Contents

List of Figures **3**

List of Tables **3**

1 Problem and Data Description **5**
1.1 Problem Formulation . . . 5
1.2 Dataset Description . . . 5
1.3 Exploratory Data Analysis . . . 6
1.3.1 Input Size and Class Distribution . . . 6
1.3.2 Representative Samples and Expected Difficulties . . . 6
1.4 Data Splitting . . . 7
1.5 Preprocessing . . . 8
1.5.1 Tensor Conversion and Normalization . . . 8
1.5.2 Model Input Representations . . . 8
1.5.3 Example Batch After Preprocessing . . . 9

2 Methodology **10**
2.1 Pipeline Overview . . . 10
2.2 Baseline and Main Methods . . . 10
2.2.1 Linear Classifier . . . 11
2.2.2 Multilayer Perceptron . . . 11
2.2.3 Convolutional Neural Network . . . 12
2.2.4 Gated Recurrent Unit . . . 12
2.2.5 Vision Transformer . . . 13
2.2.6 Shared Loss and Metrics . . . 14
2.2.7 Implementation Components . . . 14
2.3 Training Setup . . . 14
2.3.1 Shared Hyperparameters . . . 14
2.3.2 Early Stopping and Checkpoint Selection . . . 15
2.3.3 Hyperparameter Selection . . . 15
2.3.4 Experimental Environment and Training Time . . . 15
2.4 Experimental Design . . . 16
2.4.1 Performance and Computational Cost . . . 16
2.4.2 Comparison Hypotheses . . . 16

3 Implementation Results **18**
3.1 Training Behavior . . . 18
3.1.1 Validation Curves and Selected Checkpoints . . . 18
3.1.2 Generalization Gaps . . . 19
3.1.3 Implementation Issues and Fixes . . . 19
3.2 Quantitative Results . . . 20
3.2.1 Predictive Performance . . . 20
3.2.2 Model Size and Inference Cost . . . 20
3.3 Qualitative Results . . . 21
3.3.1 Correct and Incorrect Predictions . . . 21
3.4 Comparison and Discussion . . . 22

3.4.1 Representation and Inductive Bias . . . 22
3.4.2 Accuracy–Cost Trade-offs . . . 22
3.4.3 Hypothesis Assessment and Limits . . . 22
3.5 Error Analysis . . . 23
3.5.1 Class-wise Recall . . . 23
3.5.2 Frequent Confusions and Possible Improvements . . . 23
3.5.3 Error Groups and Follow-up Experiments . . . 24
3.6 Ablation Study . . . 25
3.7 Limitations and Conclusion . . . 25
3.7.1 Limits of the Evidence . . . 25
3.7.2 Conclusion and Future Work . . . 25

4 AI Usage Disclosure **27**
4.1 Website and Assignment Page . . . 27
4.2 Data Pipeline and Training Loop . . . 27
4.3 Linear, MLP, and CNN Models . . . 28
4.4 GRU and Transformer Models . . . 28
4.5 Results Provenance and Responsibility . . . 29

5 Reproducibility **30**

References **31**

List of Figures

1 Class distribution in the official training set, with 6,000 images in each of the ten classes. . . . 7
2 One representative training image from each Fashion-MNIST class, with its class ID and name. . . . 7
3 The first 16 preprocessed images from an illustrative training batch, denormalized for display and titled with their class labels. . . . 9
4 The shared workflow for training, selecting, and evaluating the five classification models. . . . 10
5 Training and validation loss and accuracy reconstructed from the five baseline epoch logs. . . . 18
6 Validation loss across epochs for the five baseline models. . . . 19
7 Correct and incorrect CNN predictions with their supplied and predicted class labels. . . . 21
8 CNN test confusion counts with true classes in rows and predicted classes in columns. . . . 24

List of Tables

1 Fashion-MNIST dataset properties, with storage size including compressed archives and extracted IDX files. . . . 6

2 The shared data split used by all five baseline models, with each class accounting for 10% of each partition. 8
3 Architecture configurations and trainable parameter counts for the five baselines. 11
4 Shared training settings used for all five architectures. 14
5 Completed epochs and total training durations for the five baseline runs. . 15
6 Comparison hypotheses and assessment criteria for the five baseline configurations. 17
7 Test performance of the five models using their best-validation-loss checkpoints. 20
8 Model size and average batched forward-pass time per test image. 21
9 Test recall percentages for each class across the five models. 23
10 CNN error groups with observed frequencies, illustrative failures, and proposed follow-up experiments. 25
11 Saved runs underlying the reported baseline results. 30

1 Problem and Data Description

1.1 Problem Formulation

Automatic clothing classification can support product categorization and catalog organization in online retail. In this assignment, we study a simplified version of this task: predicting the category of a single fashion item from a low-resolution grayscale image. The prediction unit is one image, and each image belongs to exactly one of ten mutually exclusive classes.

Let $x \in \mathbb{R}^{1\times 28\times 28}$ denote a preprocessed image and $y \in \{0, \ldots, 9\}$ its class label. A classifier f_θ maps x to ten logits $z = f_\theta(x) \in \mathbb{R}^{10}$. The predicted label is[1, PDF p. 7]

$$\hat{y} = \arg\max_k \hat{p}_k. \tag{1}$$

Here, $\hat{p}_k$ is the softmax probability of class k. The slides index classes from 1 to K; our dataset stores the corresponding IDs from 0 to 9. The parameters θ are the trainable weights and biases of f_θ.

Training minimizes the mean multiclass cross-entropy over the training set $\mathcal{D}_{\text{train}}$:[2, PDF p. 11]

$$\mathcal{L}_{\text{CE}} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik}\log p_{ik} = -\frac{1}{n}\sum_{i=1}^{n}\log p_{i,y_i}. \tag{2}$$

Here, n is the number of samples, K is the number of classes, y_{ik} is the one-hot target, and $p_{ik} = \text{softmax}(z_i)_k$. Here $K = 10$; dataset IDs are shifted by one when using the slide's class indexing. During training, the cross-entropy loss takes logits directly, so a separate softmax is unnecessary.

We compare five model families: Linear, MLP, CNN, GRU, and Transformer, to investigate how different image representations affect classification quality and computational cost under a shared data protocol. Accuracy and macro-F1 measure predictive performance, while parameter count, training time, and inference time describe resource requirements.

The main difficulty is visual ambiguity between categories such as T-shirt/top, shirt, pullover, and coat. At 28×28 resolution, fine texture and small garment details are reduced, and color information is absent. A successful classifier must therefore distinguish classes largely through silhouettes, edges, and spatial relationships.

1.2 Dataset Description

Our experiments use the official Fashion-MNIST distribution from Zalando Research, downloaded through `torchvision.datasets.FashionMNIST`. It contains 60,000 training images and 10,000 test images, with one integer annotation per image. Its ten label IDs are shown in Figure 2. The dataset is distributed under the MIT license.[9]

No dataset release/version identifier or retrieval date was recorded. The EDA notebook records the following four downloaded archives and their extracted IDX files in `FashionMNIST/raw/`:

- Training set: `train-images-idx3-ubyte.gz` and `train-labels-idx1-ubyte.gz`.
- Test set: `t10k-images-idx3-ubyte.gz` and `t10k-labels-idx1-ubyte.gz`.

The Torchvision software version reported in the experimental environment identifies the loading library, not a dataset version.

The images originate from Zalando product articles. Our analysis focuses on image properties and class distributions, using the supplied annotations without relabeling or independent verification. Fashion-MNIST is the sole dataset in this study.

Property	Value
Task	Single-label, ten-class image classification
Total samples	70,000 labeled images
Image dimensions	28×28 pixels, one grayscale channel
Raw pixel values	Unsigned 8-bit integers in $[0, 255]$
Annotations	One integer class ID in $\{0, \ldots, 9\}$ per image
Storage format	IDX image and label files, distributed as gzip archives
Recorded local size	81.85 MiB for eight downloaded and extracted files

Table 1: Fashion-MNIST dataset properties, with storage size including compressed archives and extracted IDX files.

The dataset is limited to ten broad categories and centered grayscale product images. It does not capture the full variety of clothing styles or scenes with people, clutter, occlusion, or multiple objects. Consequently, performance on this benchmark should not be interpreted as evidence of equivalent performance on unconstrained retail photographs. Class balance also does not guarantee coverage of demographic, geographic, or stylistic variation.

1.3 Exploratory Data Analysis

Before training, we examine input properties, label frequencies, representative samples, split integrity, and preprocessed batches.

1.3.1 Input Size and Class Distribution

Each input contains $28 \times 28 = 784$ pixels and has tensor shape $[1, 28, 28]$, including an explicit grayscale channel dimension. Pixel values lie in $[0, 1]$ before normalization.

Every class in the official training set contains 6,000 images, or 10% of the 60,000-image set (Figure 1). The ratio of the largest class count to the smallest is $6000/6000 = 1.00$. This balance supports training our baselines without class weighting or over/undersampling, although equal sample counts do not imply equal classification difficulty.

1.3.2 Representative Samples and Expected Difficulties

Figure 2 illustrates the ten categories using the first training image from each class. These examples allow a visual comparison of class appearances, although they do not capture all variation within each class.



Figure 1: *Class distribution in the official training set, with 6,000 images in each of the ten classes.*

Trousers and bags have distinctive shapes, while several upper-body garments share similar outlines. The low resolution makes differences such as sleeve shape, openings, and collars harder to resolve. We therefore expect more confusion among upper-body categories; confusion matrices can help assess this expectation.

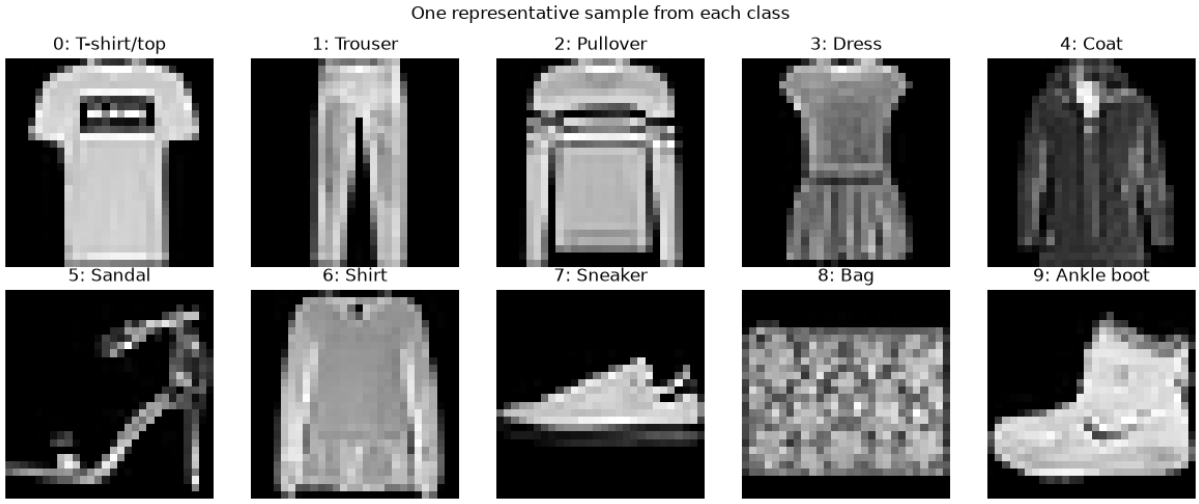


Figure 2: *One representative training image from each Fashion-MNIST class, with its class ID and name.*

1.4 Data Splitting

We divide the official 60,000-image training set into 54,000 training images and 6,000 validation images using a stratified split at the image level. Stratification preserves class proportions, and a fixed random seed of 42 ensures reproducibility. The official 10,000-image test set remains separate for final evaluation.

Split	Images	Images per class	Purpose
Training	54,000	5,400	Parameter learning
Validation	6,000	600	Checkpoint selection
Test	10,000	1,000	Final evaluation

Table 2: The shared data split used by all five baseline models, with each class accounting for 10% of each partition.

The training and validation subsets share no image indices and together cover the entire official training set. All five models use the same split and all available images, providing a common data protocol for comparing performance.

We estimate normalization statistics only from the training subset and select the model checkpoint using validation loss before evaluating it on the test set. This procedure keeps test data out of preprocessing estimation and model selection. Split checks establish separation by image index, but do not rule out visually duplicated items or shared product identities across subsets.

1.5 Preprocessing

1.5.1 Tensor Conversion and Normalization

Preprocessing converts each grayscale image to a 32-bit floating-point tensor with shape $[1, 28, 28]$ and scales raw pixel values from $[0, 255]$ to $[0, 1]$. Channel normalization uses the training mean and standard deviation:[3, PDF p. 15]

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j}. \tag{3}$$

Here, j indexes a feature. For our images, we apply the same channel statistics to each pixel scaled to $[0, 1]$. The experiment estimates $\mu_{\text{train}} = 0.286209$ and $\sigma_{\text{train}} = 0.353160$ from all pixels in the 54,000-image training subset; these numerical values are experimental results, not slide values. These statistics remain fixed for validation and test images. After normalization, pixel values lie approximately in $[-0.810, 2.021]$.

For this baseline comparison, we preserve the original 28×28 image dimensions and supplied labels, limiting preprocessing to pixel scaling and normalization. This excludes resizing, padding, augmentation, and additional image cleaning. Checks before training confirm batch dimensions, finite pixel values, and label IDs in the range from 0 to 9.

1.5.2 Model Input Representations

All five models receive the same normalized images in representations suited to their architectures. Linear and MLP use 784 flattened features, while CNN retains the channel and spatial dimensions. GRU treats the 28 image rows as a sequence of 28 timesteps, each with 28 features. Transformer uses 49 non-overlapping 4×4 patches, each containing 16 pixels before embedding. The data split and normalization remain identical across these representations.

1.5.3 Example Batch After Preprocessing

Training uses a batch size of 128, giving a full image batch of shape $[128, 1, 28, 28]$ and an integer label vector of shape [128]. Training images are shuffled, while validation and test images retain their order. Incomplete final batches are retained so that every image is included.

For exploratory analysis, we inspect a separate batch of 64 images with image shape $[64, 1, 28, 28]$, label shape [64], mean 0.046, and standard deviation 1.004. A single batch need not have exactly zero mean and unit variance because normalization statistics describe the complete training subset. Figure 3 displays its first 16 images after reversing normalization for visualization; training inputs remain normalized.

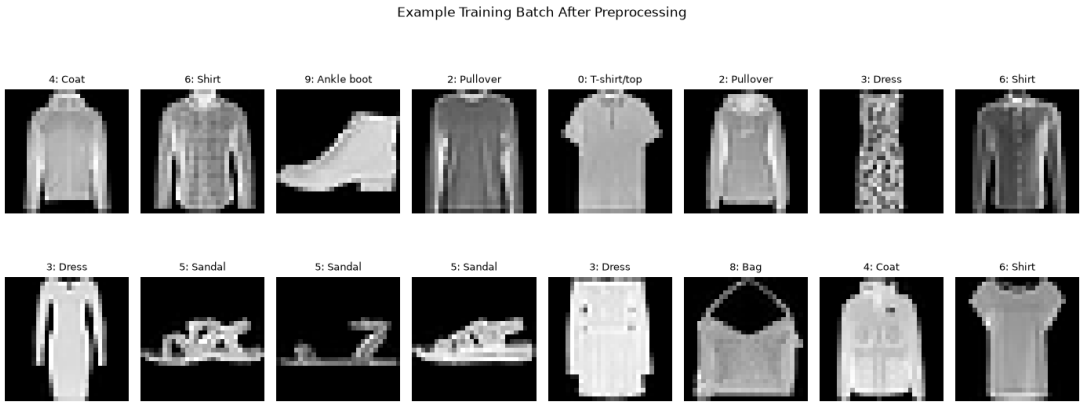


Figure 3: *The first 16 preprocessed images from an illustrative training batch, denormalized for display and titled with their class labels.*

2 Methodology

2.1 Pipeline Overview

We compare five architectures under the same data and training protocol to study how their representations influence classification performance and computational cost. Figure 4 summarizes the workflow. The split and normalization follow Section 1; only the architecture and its input representation change between baselines.

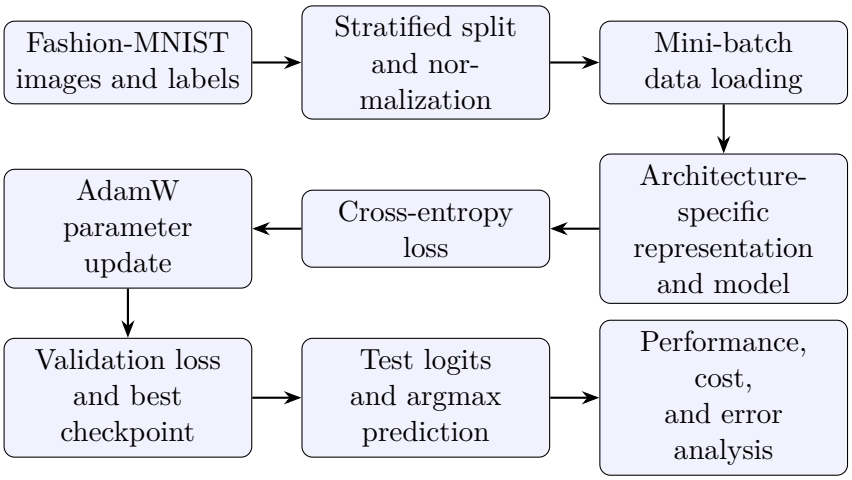


Figure 4: *The shared workflow for training, selecting, and evaluating the five classification models.*

During each training batch, the model produces ten logits, cross-entropy measures prediction error, and backpropagation supplies gradients for the optimizer. Validation follows each epoch without parameter updates. For final evaluation, the selected checkpoint produces logits for the held-out test images, and argmax converts them to class IDs. No additional prediction post-processing is required.

2.2 Baseline and Main Methods

The Linear classifier provides a simple reference, while MLP, CNN, GRU, and Transformer introduce nonlinear, spatial, recurrent, and attention-based representations. Table 3 lists the configurations and trainable parameter counts. All models are trained from scratch without pretrained weights.

Model	Architecture summary	Parameters
Linear	Flatten; $784 \to 10$	7,850
MLP	$784 \to 256 \to 128 \to 10$; ReLU; dropout 0.2	235,146
CNN	Two convolution blocks with 32 and 64 channels; adaptive pooling; $256 \to 128 \to 10$ head	53,098
GRU	28 row timesteps; 28 input features; one unidirectional layer with 128 hidden units; $128 \to 10$ head	61,962
Transformer	49 patches; 128-dimensional tokens; four encoder layers with four attention heads; $128 \to 10$ head	803,338

Table 3: Architecture configurations and trainable parameter counts for the five baselines.

2.2.1 Linear Classifier

Each image is flattened into a vector $v \in \mathbb{R}^{784}$. An affine mapping produces class logits:[1, PDF p. 19]

$$\mathbf{y} = W\mathbf{x} + \mathbf{b} \in \mathbb{R}^M. \tag{4}$$

Here, $\mathbf{x} \in \mathbb{R}^N$, $W \in \mathbb{R}^{M\times N}$, and $\mathbf{b} \in \mathbb{R}^M$. For our classifier, $N = 784$, $M = 10$, $\mathbf{x} = v$, and the layer output $\mathbf{y}$ contains logits (it is not the ground-truth label). Softmax converts these logits to class probabilities when needed, but is incorporated into the cross-entropy calculation during training.[4, PDF p. 15] The layer has $M \times N$ weights plus M biases.[1, PDF p. 19] Substituting our dimensions gives $784 \times 10 + 10 = 7{,}850$ parameters. It offers a compact reference for evaluating the benefit of more complex representations. Its main limitation is the linear decision boundary: it cannot explicitly learn nonlinear combinations of garment features or local spatial patterns.

2.2.2 Multilayer Perceptron

The MLP also receives 784 flattened pixels, but uses two hidden layers with 256 and 128 units. The activation is $\mathrm{ReLU}(z) = \max(0, z)$.[1, PDF p. 26] Here, z is a scalar preactivation: negative values become zero, while nonnegative values remain unchanged. We use $\phi = \mathrm{ReLU}$ in the forward mapping below. A dropout layer with probability 0.2 follows the second ReLU, and the final affine layer produces ten logits.

The hidden-layer mapping is[1, PDF p. 15]

$$\mathbf{h}^{(l)} = \phi\Big(W^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\Big), \qquad l = 1, \ldots, L. \tag{5}$$

The slide sets $\mathbf{h}^{(0)} = \mathbf{x}$. Applying this rule and adding our chosen dropout gives the following configuration:

$$h_1 = \phi(W_1 v + b_1), \qquad h_2 = \phi(W_2 h_1 + b_2), \qquad z = W_3\,\mathrm{Dropout}_{0.2}(h_2) + b_3. \tag{6}$$

Here, $v \in \mathbb{R}^{784}$ is the flattened image, $h_1 \in \mathbb{R}^{256}$ and $h_2 \in \mathbb{R}^{128}$ are hidden activations, and $z \in \mathbb{R}^{10}$ contains class logits. The learned weight matrices have shapes $W_1 \in \mathbb{R}^{256\times 784}$, $W_2 \in \mathbb{R}^{128\times 256}$, and $W_3 \in \mathbb{R}^{10\times 128}$, with matching bias vectors.

Dropout randomly removes hidden activations during training to reduce dependence on individual features; it is disabled during evaluation. Compared with Linear, the MLP

can represent more complex decision boundaries. However, flattening provides no explicit preference for neighboring pixels or shared patterns at different image locations, and its dense connections account for most of its 235,146 parameters.

2.2.3 Convolutional Neural Network

A convolution computes each output feature from a local input region using shared filter weights:[5, PDF p. 5]

$$Y_k = \sum_{c=1}^{C_{\text{in}}} X_c \star W_{k,c} + b_k. \tag{7}$$

Here X_c is input channel c, $W_{k,c}$ is a filter, Y_k is output channel k, and $\star$ denotes local correlation. Our convolutions omit b_k. In the output-size formula below, k instead denotes kernel size, s is stride, and p is padding. For unit dilation, the output size is:[5, PDF p. 27]

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - k}{s} \right\rfloor + 1, \qquad W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2p - k}{s} \right\rfloor + 1. \tag{8}$$

Here, H_{in} and W_{in} are the input height and width, and $\lfloor \cdot \rfloor$ denotes rounding down to an integer.

Our CNN retains the two-dimensional image structure and contains two feature-extraction blocks. Each block uses a 3×3 convolution with padding 1, batch normalization, ReLU, 2×2 max pooling, and spatial dropout. The convolutions have 32 and 64 output channels, respectively, and omit bias because batch normalization follows them. The following shape sequence is calculated for our configuration:

$$1 \times 28 \times 28 \to 32 \times 28 \times 28 \to 32 \times 14 \times 14 \to 64 \times 14 \times 14 \to 64 \times 7 \times 7. \tag{9}$$

Each shape follows the order channels $\times$ height $\times$ width and omits the batch dimension. Arrows denote successive convolution or pooling stages: convolutions change channel count while preserving spatial size, and each pooling stage halves height and width. Convolution learns local features with weights shared across image positions, while each feature-map channel records the response to a learned filter. Max pooling reduces spatial resolution by retaining the strongest activation in each neighborhood. These operations favor local patterns such as edges and garment contours and provide some tolerance to small spatial shifts.

Spatial dropout probabilities are 0.1 and 0.2 after the first and second pooling stages. Adaptive average pooling then reduces each of the 64 feature maps to 2×2, producing 256 features. The classification head uses 128 hidden units with ReLU and dropout 0.3 before the ten-logit output. This design has 53,098 parameters, though spatial downsampling can discard fine details needed to distinguish similar garments.

2.2.4 Gated Recurrent Unit

The GRU equations[6, PDF p. 32] use $\boldsymbol{\xi}_t = [\mathbf{h}_{t-1}; \mathbf{x}_t] \in \mathbb{R}^{H+D}$:

$$\mathbf{r}_t = \sigma(W_r \boldsymbol{\xi}_t + \mathbf{b}_r), \qquad \mathbf{z}_t = \sigma(W_z \boldsymbol{\xi}_t + \mathbf{b}_z), \tag{10}$$

$$\tilde{\mathbf{h}}_t = \tanh(W_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_h)\,, \tag{11}$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t + \mathbf{z}_t \odot \mathbf{h}_{t-1}. \tag{12}$$

Here $\mathbf{r}_t$ and $\mathbf{z}_t$ are reset and update gates, σ is sigmoid, $\odot$ is elementwise multiplication, and brackets with a semicolon denote concatenation. These are the lecture's compact equations; the experiment uses PyTorch's GRU implementation.

We treat each image as a top-to-bottom sequence of 28 rows. At timestep t, the input $r_t \in \mathbb{R}^{28}$ contains the pixels of row t (this row symbol is distinct from the slide's reset gate $\mathbf{r}_t$). A single unidirectional GRU has hidden size 128. Our model is summarized by:

$$h_t = \mathrm{GRU}(r_t, h_{t-1}), \qquad z = W h_{28} + b. \tag{13}$$

Here, $t \in \{1, \ldots, 28\}$ is the row timestep, r_t contains its 28 pixels, and $h_{t-1}, h_t \in \mathbb{R}^{128}$ are the previous and updated hidden states. The initial state h_0 is zero. The function GRU denotes the learned gated state update, and h_{28} summarizes the final timestep. The head weights $W \in \mathbb{R}^{10\times128}$ and bias $b \in \mathbb{R}^{10}$ produce logits $z \in \mathbb{R}^{10}$. Reset and update gates control how previous information contributes to the new hidden state. The final state summarizes the sequence and feeds a linear ten-class head. This configuration has 61,962 parameters and uses no recurrent dropout.

The sequential representation can capture relationships between rows without flattening the entire image into one dense input. However, the reading direction is imposed on a two-dimensional image, processing is sequential across rows, and information must be retained in the hidden state until the last timestep. These constraints differ from the local spatial structure favored by CNN.

2.2.5 Vision Transformer

Each image is divided into 49 non-overlapping 4×4 patches. A learned projection maps the 16 pixels in each patch to a 128-dimensional token. A learnable classification token is prepended, and learned positional embeddings identify the locations of all 50 tokens. Without positional information, attention alone would not distinguish patch order.

The token sequence enters four Transformer encoder layers, each with four attention heads. For one head, projected token features form queries Q, keys K, and values V. Attention is computed as:[7, PDF p. 11]

$$A = \mathrm{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}}\right), \qquad Y = AV. \tag{14}$$

For our configuration, $d_k = 128/4 = 32$. For one image and one head, $Q, K, V \in \mathbb{R}^{50\times32}$ are learned projections of the 50 tokens. The superscript T denotes transpose, so QK^{T} contains pairwise query–key scores. Row-wise softmax gives attention weights $A \in \mathbb{R}^{50\times50}$, and $Y = AV \in \mathbb{R}^{50\times32}$ contains weighted value vectors. In the batched shape $[B, 50, 128]$, B denotes the number of images in the batch. The attention weights describe how strongly each token draws information from every other token. Four head outputs are concatenated and projected back to 128 dimensions, preserving the sequence shape $[B, 50, 128]$. Each encoder layer combines attention with residual connections and a feed-forward network of width 512 using GELU activation. Layer normalization precedes the attention and feed-forward sublayers, and dropout is 0.1.

After the final encoder layer, the classification-token representation undergoes layer normalization and a linear mapping to ten logits. Attention allows interactions between distant image regions, but this model has 803,338 parameters and a weaker built-in preference for local image structure than CNN. Under a small-image training budget, its additional capacity need not translate into better accuracy.

2.2.6 Shared Loss and Metrics

All five models output ten logits and minimize the same mean multiclass cross-entropy defined in Section 1. The loss compares the logit-derived class probabilities with the supplied labels, without a separate softmax before loss calculation. Predictions use the class with the largest logit. Accuracy and macro-F1 measure overall correctness and class-wise performance, while trainable parameter count, training time, and inference time describe computational cost. Confusion matrices and prediction examples support error analysis; metric definitions and timing procedures appear in Section 2.4.

2.2.7 Implementation Components

We implement the model composition and forward-pass workflow, row-sequence conversion, patch-token assembly, data preparation, training and validation loops, checkpoint selection, and evaluation orchestration. These components use PyTorch's standard layers and automatic differentiation: affine and convolutional layers, pooling, activations, normalization, dropout, GRU, and Transformer encoder layers come from PyTorch, as do cross-entropy and AdamW. In particular, the GRU cell and Transformer encoder internals are library implementations. Torchvision supplies Fashion-MNIST access and tensor transforms, while scikit-learn supplies stratified splitting, macro-F1, and confusion-matrix calculations. Our contribution is the architecture configuration and integration of these components into a shared experimental workflow, rather than reimplementing each underlying operation.

2.3 Training Setup

2.3.1 Shared Hyperparameters

Setting	Value
Loss	Mean multiclass cross-entropy
Optimizer	AdamW
Learning rate	10^{-3}, constant throughout training
Weight decay	10^{-4}
Batch size	128
Maximum epochs	20
Early stopping	Patience 5; minimum improvement 10^{-4}
Checkpoint selection	Lowest validation loss
Random seed	42
Precision	Standard 32-bit floating point; no mixed precision

Table 4: Shared training settings used for all five architectures.

AdamW combines adaptive gradient updates with decoupled weight decay. The learning rate remains fixed without a scheduler. Weight decay applies to all five models;

dropout provides additional architecture-specific regularization in MLP, CNN, and Transformer. Validation runs in evaluation mode, with dropout disabled and CNN batch-normalization statistics fixed.

2.3.2 Early Stopping and Checkpoint Selection

The checkpoint is saved whenever validation loss reaches a new minimum. Early stopping uses a separate criterion: training ends after five consecutive epochs without a loss reduction of at least 10^{-4} relative to the previous best loss. Therefore, a smaller improvement can update the checkpoint without resetting the stopping counter. Test evaluation always reloads the checkpoint with the lowest validation loss, rather than using the final training state.

2.3.3 Hyperparameter Selection

The comparison uses one fixed baseline configuration per model and a shared training budget, rather than an exhaustive hyperparameter search. Architecture widths and regularization differ as listed above; parameter counts and computational budgets are not matched. This design compares practical baseline configurations under a common protocol, rather than isolating architecture independently of model capacity.

2.3.4 Experimental Environment and Training Time

The recorded experimental environment includes an NVIDIA GeForce RTX 5070 Ti, Python 3.12.10, PyTorch 2.14.0+cu130, Torchvision 0.29.0+cu130, and scikit-learn 1.9.0. These software versions were recorded after the baseline runs, so they describe the available environment rather than a separately captured environment for each run. Training duration is measured around the complete training loop, including validation and checkpoint saving. Table 5 reports elapsed time and completed epochs; early stopping means that the runs do not all complete the same number of epochs.

Model	Completed epochs	Training time (s)
Linear	17	290.77
MLP	15	257.38
CNN	20	343.46
GRU	18	305.00
Transformer	20	362.18

Table 5: Completed epochs and total training durations for the five baseline runs.

2.4 Experimental Design

2.4.1 Performance and Computational Cost

Accuracy measures the proportion of correctly classified test images. We compute it as

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}(\hat{y}_i = y_i), \tag{15}$$

Here, N is the number of test images, and the indicator equals one when true label y_i matches predicted label $\hat{y}_i$. This sample-based multiclass accuracy expression is our evaluation definition; the lecture accuracy formula is written for binary classification.[8, PDF p. 13]

Precision, recall, and F1 are defined as:[8, PDF p. 13]

$$\text{Precision} = \frac{TP}{TP+FP}, \qquad \text{Recall} = \frac{TP}{TP+FN}, \qquad \text{F1} = 2\cdot\frac{\text{Precision}\cdot\text{Recall}}{\text{Precision}+\text{Recall}}. \tag{16}$$

We compute these per class using a one-versus-rest interpretation: TP, FP, and FN count true positives, false positives, and false negatives for that class. The macro-average is:[8, PDF p. 19]

$$\text{MacroF1} = \frac{1}{K}\sum_{k=1}^{K}\text{F1}_k. \tag{17}$$

Here $K = 10$, and the slide's class index $k = 1, \ldots, K$ corresponds to dataset IDs 0 through 9. Each class receives equal weight. An undefined class F1 contributes zero. Together, these metrics describe overall correctness and performance across categories. Confusion matrices use true classes as rows and predicted classes as columns, making systematic class confusions visible. Training and validation curves show learning behavior, while correctly and incorrectly classified examples support qualitative error analysis.

Computational cost includes trainable parameter count, total training time, and average inference time per image. Inference timing follows one warm-up batch and measures only batched model forward passes, excluding data loading, device transfer, and metric calculation. CUDA synchronization bounds each timed forward pass. Total forward-pass time divided by the number of test images gives the reported per-image average; this is batched throughput expressed per image, rather than single-image request latency.

2.4.2 Comparison Hypotheses

Our central hypothesis is that a representation suited to image structure can improve classification beyond a linear mapping, while increased capacity alone may not justify its computational cost. Table 6 states the comparisons and the evidence used to assess them.

Comparison	Hypothesis and changed factor	Assessment criterion
Linear vs. MLP	Nonlinear hidden layers improve classification of flattened pixels.	Higher test accuracy and macro-F1, considered alongside added parameters and timing.
MLP vs. CNN	Local connectivity and shared filters improve predictive performance over dense flattened features.	Support if CNN has higher accuracy and macro-F1; assess parameter efficiency and class errors separately.
CNN vs. GRU	Row-wise recurrence offers a different balance of predictive quality and inference cost from spatial convolution.	Compare accuracy and macro-F1 jointly with inference time; lower cost with lower quality is a trade-off, not overall superiority.
CNN vs. Transformer	Greater capacity and global patch attention do not necessarily improve predictive performance over CNN.	If Transformer fails to exceed CNN on both predictive metrics, greater capacity has no demonstrated predictive advantage in this run; compare cost separately.

Table 6: Comparison hypotheses and assessment criteria for the five baseline configurations.

Across comparisons, the fixed factors are dataset split, normalization, batch size, optimizer, learning rate, weight decay, epoch limit, stopping rule, checkpoint criterion, seed, and evaluation procedure. Architecture, representation, capacity, and architecture-specific regularization vary together, so these comparisons are not single-factor ablations. A predictive advantage requires higher accuracy and macro-F1; conflicting metrics are reported as mixed evidence. Lower parameter count or runtime indicates an advantage only in that resource measure, and quality–cost trade-offs are described without a single combined score. With one run per model at seed 42, these criteria describe observed differences rather than statistical significance; repeated runs would be needed to assess variability and the reliability of small performance gaps.

3 Implementation Results

The following results compare the five baseline configurations on the same 10,000-image test set. Each model is evaluated using its lowest-validation-loss checkpoint, with the training and evaluation protocol described in Section 2. We report the observed results from one run per model at seed 42.

3.1 Training Behavior

3.1.1 Validation Curves and Selected Checkpoints

Figure 5 plots training and validation loss and accuracy from the recorded epoch logs. Training accuracy increases for all five models, while validation improvement slows earlier for Linear, MLP, and GRU. The separation between the training and validation trajectories helps distinguish continued fitting of training images from improvement on held-out images.

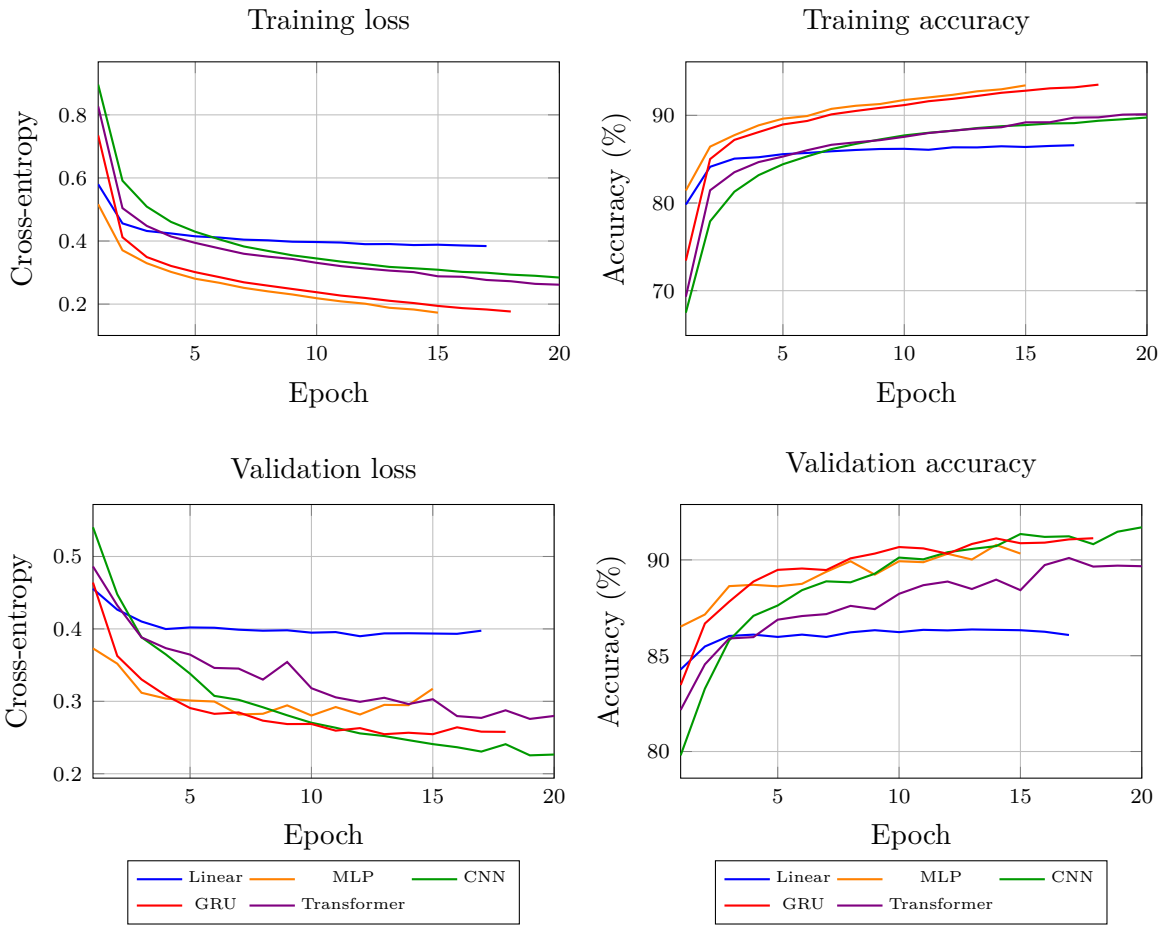


Figure 5: *Training and validation loss and accuracy reconstructed from the five baseline epoch logs.*

Figure 6 shows rapid loss reduction during the first few epochs, followed by slower improvement or fluctuations. Linear levels off near 0.39, while the nonlinear models reach lower validation losses. CNN continues improving toward the end of the training budget

and reaches the lowest recorded validation loss, 0.2254, at epoch 19. The legend label "rnn" denotes the GRU baseline.

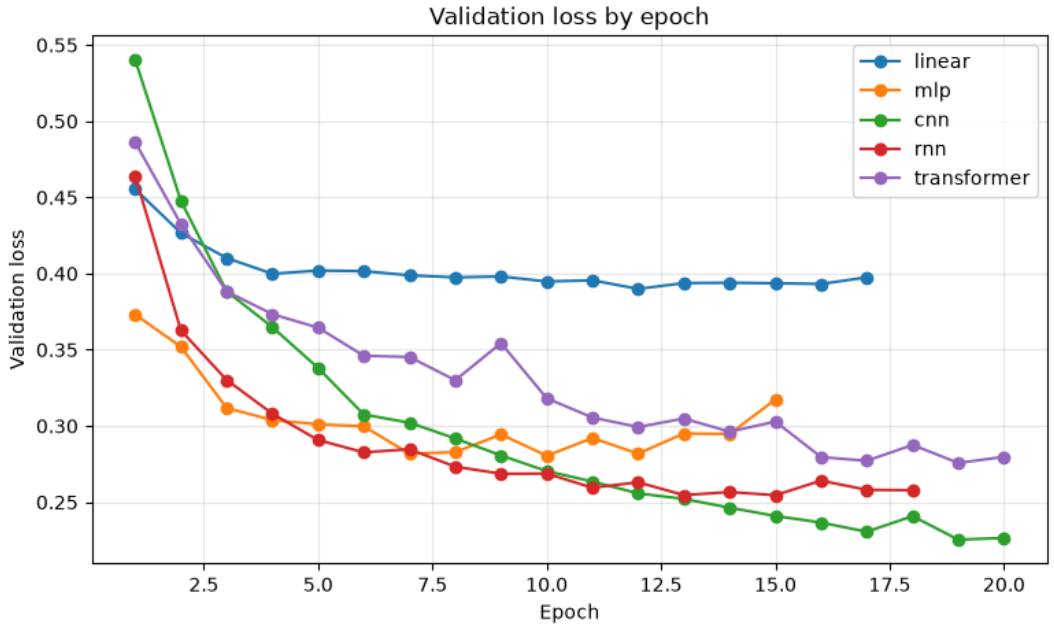


Figure 6: *Validation loss across epochs for the five baseline models.*

The selected epochs are 12 for Linear, 10 for MLP, 19 for CNN, 15 for GRU, and 19 for Transformer. Linear, MLP, and GRU stop after 17, 15, and 18 epochs, respectively, while CNN and Transformer complete all 20 epochs. The selected epoch can precede the final epoch because evaluation uses the best checkpoint rather than the last state. Training durations range from 257.38 seconds for MLP to 362.18 seconds for Transformer, as reported in Table 5.

3.1.2 Generalization Gaps

At the final epoch, MLP has training loss 0.1728 versus validation loss 0.3174, and GRU has training loss 0.1765 versus validation loss 0.2578. Their lower training losses and later validation fluctuations suggest overfitting, supporting the use of earlier checkpoints. Linear has a smaller final loss gap, 0.3840 versus 0.3975, but lower test accuracy than every nonlinear model; this pattern is consistent with limited representational capacity.

CNN has final training loss 0.2842 and validation loss 0.2265. A lower validation loss here does not imply leakage: training includes active dropout and batch-normalization updates, whereas validation uses evaluation mode. Transformer ends with training loss 0.2615 and validation loss 0.2798, with fluctuations visible in its validation curve. These final-epoch losses describe learning behavior and are distinct from the selected-checkpoint test losses below.

3.1.3 Implementation Issues and Fixes

During development, a pytest module-name collision required reorganizing the test packages so that test collection could distinguish the modules. The automated test environment was also changed from Python 3.11 to Python 3.12 to align with the project's

declared runtime. These changes are recorded in the development history; they concern test execution and reproducibility, rather than a measured improvement in classification accuracy.

The final workflow checks split separation, input dimensions, finite tensor values, and valid class IDs before training. Evaluation reloads the selected checkpoint and disables gradient calculation; GPU timing uses synchronization to avoid measuring only asynchronous launch time. These checks address data and measurement validity, but do not constitute a complete record of every implementation incident.

3.2 Quantitative Results

3.2.1 Predictive Performance

Table 7 reports held-out test loss, accuracy, and macro-F1. CNN correctly classifies 9,023 of the 10,000 test images and achieves the highest accuracy and macro-F1 among these configurations. GRU is second on both metrics, followed by Transformer, MLP, and Linear. The similar ordering of accuracy and macro-F1 indicates that CNN's overall advantage is also reflected in the equally weighted class-wise score.

Model	Test loss	Accuracy (%)	Macro-F1
Linear	0.4455	84.23	0.8395
MLP	0.3318	88.56	0.8855
CNN	0.2691	90.23	0.9023
GRU	0.2926	89.72	0.8970
Transformer	0.3015	89.16	0.8908

Table 7: Test performance of the five models using their best-validation-loss checkpoints.

CNN improves accuracy over Linear by 6.00 percentage points, over MLP by 1.67 points, over GRU by 0.51 points, and over Transformer by 1.07 points. These are observed differences from single runs, rather than estimates of average performance over multiple seeds. Mean and standard deviation are therefore not reported.

3.2.2 Model Size and Inference Cost

Table 8 complements predictive performance with parameter count and batched inference time per image. Linear has the smallest model and fastest measured inference. Transformer has the most parameters and slowest inference, while CNN achieves higher accuracy with substantially fewer parameters.

Model	Trainable parameters	Inference (ms/image)
Linear	7,850	0.0013
MLP	235,146	0.0022
CNN	53,098	0.0070
GRU	61,962	0.0033
Transformer	803,338	0.0232

Table 8: Model size and average batched forward-pass time per test image.

Parameter count alone does not predict runtime: CNN has fewer parameters than MLP but a slower forward pass because convolution processes spatial feature maps. GRU gives up 0.51 accuracy points relative to CNN while requiring less than half its measured inference time. Training duration also depends on completed epochs, regularization, and validation overhead, so the elapsed times do not compare equal amounts of training work.

3.3 Qualitative Results

3.3.1 Correct and Incorrect Predictions

Figure 7 shows five correct and five incorrect CNN predictions, with both supplied and predicted labels. These are the first examples in each correctness group in test-set order; they are illustrative rather than a random or difficulty-ranked sample.

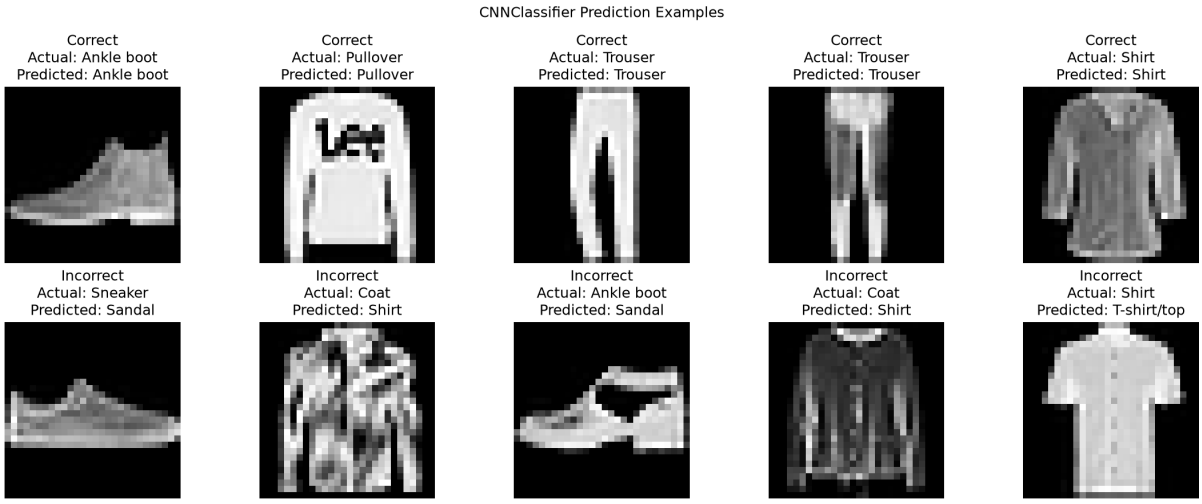


Figure 7: *Correct and incorrect CNN predictions with their supplied and predicted class labels.*

The correctly classified ankle boot and trousers have distinctive silhouettes. The correct shirt in the upper-right panel is a useful example of success in a difficult category: CNN misses 28.3% of all test shirts, yet recognizes this garment with its visible collar and front opening. The correct pullover also illustrates successful discrimination within the frequently confused upper-body group. Here, "difficult" refers to the category-level error pattern; these images were not ranked by confidence or individual difficulty.

The failure examples include two coats predicted as shirts and a shirt predicted as T-shirt/top, illustrating ambiguity among upper-body categories. A sneaker and an ankle

boot are predicted as sandals, showing that individual footwear images can also be misleading despite high aggregate footwear recall. At this resolution, outlines and openings may resemble those of another category; this is a plausible visual explanation, rather than a verified account of the model's internal reasoning.

3.4 Comparison and Discussion

3.4.1 Representation and Inductive Bias

The Linear-to-MLP comparison supports the usefulness of nonlinear feature combinations: accuracy increases by 4.33 percentage points, and macro-F1 rises from 0.8395 to 0.8855. This gain comes with a much larger parameter count, so it does not establish equal-cost superiority.

CNN improves on MLP while using fewer parameters, consistent with the hypothesis that local connectivity and weight sharing are useful for small clothing images. GRU also outperforms MLP, suggesting that a row-sequence representation can capture useful relationships. Neither comparison isolates representation alone, since architecture size and regularization also change.

Transformer does not surpass CNN despite having about 15.1 times as many parameters. Under this training budget, global patch attention and greater capacity do not produce the best observed predictive performance. This result does not establish that attention is unsuitable for the task: patch size, model size, optimization settings, and available training duration may all affect the outcome.

3.4.2 Accuracy–Cost Trade-offs

For the tested configurations, CNN provides the strongest predictive performance, Linear prioritizes compactness and speed, and GRU offers a competitive accuracy–inference-cost trade-off. MLP remains faster at inference than CNN, although its accuracy is lower and its parameter count is higher. Transformer is the most costly model by both parameter count and measured inference time without a corresponding accuracy advantage. The preferred model therefore depends on whether predictive quality, memory, or throughput is the primary requirement.

3.4.3 Hypothesis Assessment and Limits

The Linear–MLP hypothesis is supported by increases of 4.33 accuracy points and 0.0460 macro-F1. The MLP–CNN hypothesis is also supported: CNN gains 1.67 accuracy points and 0.0168 macro-F1 while using about 77.4% fewer parameters. The CNN–GRU comparison shows the expected trade-off: GRU loses 0.51 accuracy points and 0.0053 macro-F1 but reduces measured inference time by about 52.9%. The CNN–Transformer comparison supports the caution about capacity: Transformer loses 1.07 accuracy points and 0.0115 macro-F1 while requiring about 15.1 times the parameters and 3.3 times the inference time.

These differences are consistent with the architectural explanations above, but do not demonstrate that spatial bias alone caused CNN's advantage. All models use the same loss and training protocol, while widths, representations, and dropout settings differ. The small grayscale images may favor local contour features, and one shared optimization

budget may suit the architectures differently. Without matched-capacity experiments, controlled ablations, and repeated seeds, the results cannot establish a universal ranking or statistically reliable superiority.

3.5 Error Analysis

3.5.1 Class-wise Recall

Table 9 compares recall for every category. Because each class contains 1,000 test images, recall percentages also directly indicate the fraction of its examples correctly classified. Shirt has the lowest recall for all five models, supporting the ambiguity anticipated in Section 1.3.

Class	Linear	MLP	CNN	GRU	Transformer
T-shirt/top	81.8	85.9	85.5	82.9	87.2
Trouser	95.9	96.9	97.8	97.3	98.7
Pullover	75.3	75.7	83.4	79.1	88.5
Dress	89.8	87.1	89.1	92.0	87.6
Coat	75.0	87.5	86.1	86.2	81.1
Sandal	91.8	94.7	97.9	97.4	95.5
Shirt	50.9	68.7	71.7	71.8	64.8
Sneaker	95.4	96.2	96.4	97.3	96.1
Bag	94.2	97.5	98.3	97.8	96.9
Ankle boot	92.2	95.4	96.1	95.4	95.2

Table 9: Test recall percentages for each class across the five models.

The strongest overall model is not strongest in every class. Transformer has the highest trouser and pullover recall, GRU leads on dress and shirt, MLP leads on coat, and CNN leads on sandal and bag. These differences motivate examining class-specific errors rather than selecting a model from overall accuracy alone.

3.5.2 Frequent Confusions and Possible Improvements

Figure 8 shows the CNN confusion matrix. Its 977 test errors include 283 missed shirts, so shirt false negatives account for about 29.0% of all errors. Among the 1,000 true shirts, 114 are predicted as T-shirt/top and 81 as coat, corresponding to 11.4% and 8.1% of that class. In the opposite direction, 96 T-shirt/top images and 63 coats are predicted as shirts. Similar silhouettes and limited collar or sleeve detail are plausible causes.

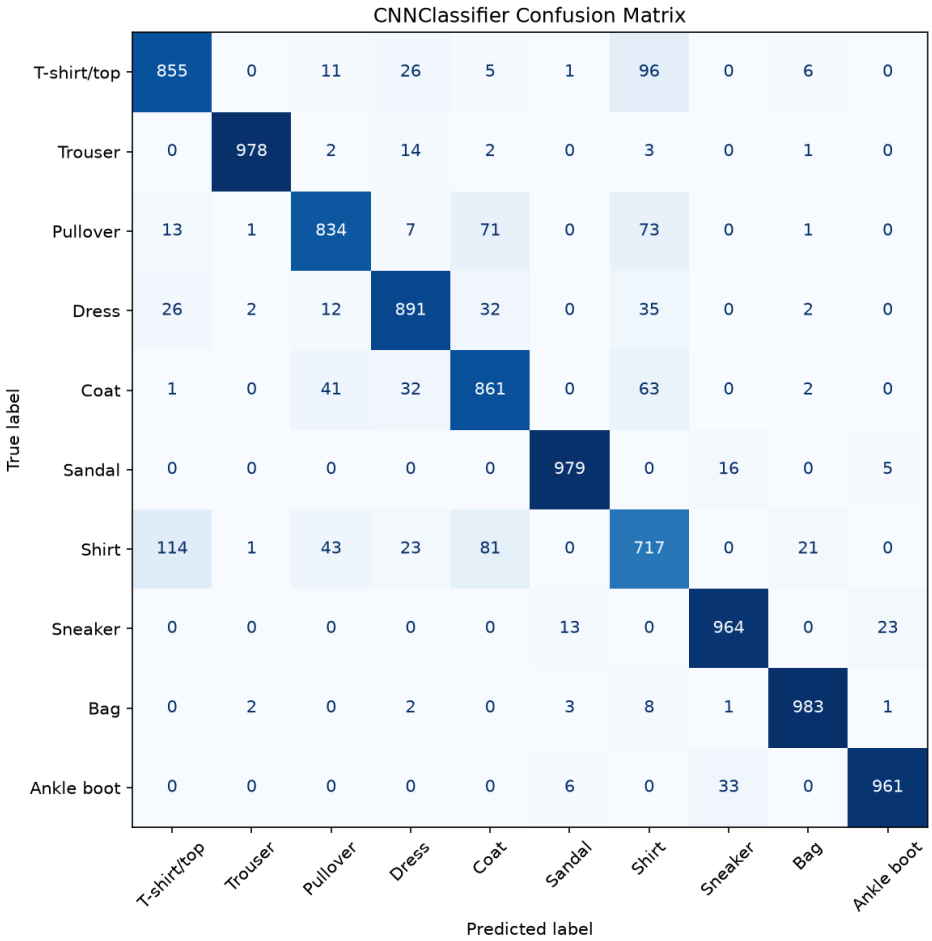


Figure 8: *CNN test confusion counts with true classes in rows and predicted classes in columns.*

Pullover confusion also contributes substantially: 73 pullovers are predicted as shirts and 71 as coats, representing 7.3% and 7.1% of true pullovers. Footwear errors are less frequent but still systematic: 33 ankle boots are predicted as sneakers, and 23 sneakers as ankle boots. These rates are 3.3% and 2.3% of their respective true classes.

Possible follow-up experiments include reducing CNN downsampling to preserve garment details, testing modest image augmentation, and inspecting ambiguous annotations before changing the training targets. Upper-body errors deserve particular attention because class frequencies are balanced and additional class weighting would not directly resolve visual ambiguity. Any proposed modification should be selected on validation data and evaluated with a controlled comparison; no improvement from these ideas is claimed in the present results.

3.5.3 Error Groups and Follow-up Experiments

Table 10 connects the two main error groups to their observed frequencies, visual examples, possible causes, and proposed improvements. Counts and rates refer to CNN, and each percentage uses the 1,000 test images in the stated true class as its denominator. The example panel is illustrative; it does not provide the frequency estimates, which come from the full confusion matrix.

Error group	Frequency and examples	Possible cause and improvement
Upper-body garments	Shirt → T-shirt/top: 114 (11.4%); shirt → coat: 81 (8.1%); pullover → shirt: 73 (7.3%). Figure 7 shows a shirt predicted as T-shirt/top and two coats predicted as shirts.	Similar outlines and weak collar/sleeve detail may obscure category boundaries; test less spatial downsampling and inspect ambiguous examples before considering label corrections.
Footwear	Ankle boot → sneaker: 33 (3.3%); sneaker → ankle boot: 23 (2.3%). Figure 7 also shows a sneaker and an ankle boot predicted as sandals.	Side-view silhouettes and openings may resemble another footwear class; test modest translations or rotations to assess sensitivity to appearance variation.

Table 10: CNN error groups with observed frequencies, illustrative failures, and proposed follow-up experiments.

3.6 Ablation Study

No separate ablation was performed in these baseline runs. The five-model comparison changes architecture, representation, capacity, and regularization together, so it cannot attribute a gain to one component in isolation. A useful follow-up would compare the same CNN with and without spatial dropout, holding its split, optimizer, width, and training budget fixed. Additional comparisons could vary GRU reading direction or Transformer patch size separately. These are proposed experiments, not completed results.

3.7 Limitations and Conclusion

3.7.1 Limits of the Evidence

One run per model does not measure seed-to-seed variability, and the 0.51-point gap between CNN and GRU is not evidence of a statistically reliable architecture ranking. The configurations are fixed baselines without extensive tuning or matched parameter budgets. Timing values are specific to the measured hardware and batching procedure, and total training time includes different numbers of epochs. The findings also concern centered 28×28 grayscale Fashion-MNIST images rather than unconstrained clothing photographs.

3.7.2 Conclusion and Future Work

Under the shared protocol, CNN achieves the highest observed accuracy and macro-F1 with substantially fewer parameters than Transformer. Linear provides the smallest and fastest reference, while GRU offers competitive accuracy with faster measured inference than CNN. These results support considering representation and structural bias alongside capacity and computational cost. Confusion among shirts, T-shirt/top, pullovers, and coats remains the main classification challenge. Repeated runs, controlled ablations, and

validation-based tuning would help assess the stability of the comparison and identify which changes improve those difficult categories.

4 AI Usage Disclosure

This section summarizes entries A1-1–A1-4 in AI_USAGE.md. Prompt examples are representative summaries, not verbatim transcripts. Code, configuration, test, and notebook paths below are relative to `assignment1/`; website files are relative to the repository root.

4.1 Website and Assignment Page

Tool / model	Codex (OpenAI), GPT-5.6
Member / verifier	Tran Manh Thang (2353124)
Stage	September 2026, M1 Draft website build

- **Purpose and scope:** Draft website structure and the Problem Statement, Dataset & EDA, and Methodology content. Files: `index.html`, `assignment1.html`, `assignment2.html`, `assignment3.html`, `styles.css`, `README.md`, and `assets/eda/`.
- **Edits:** Adapted the layout and descriptions to the implemented pipeline and EDA outputs; revised figure references, captions, and deliverable links.
- **Verification:** Compared content with Handbook Sections 2.1–2.2 and 3.1–3.2 (revision 14 September 2026), repository code, `notebooks/01_eda.ipynb`, and `m1_draft_results/` JSON outputs. Inspected rendered pages, figures, and links.

Representative prompt example

> "Draft the landing page and assignment-page structure. Use the repository code and exported EDA notebook to describe the problem, dataset, and methodology, and keep all figures and numbers consistent with the saved outputs."

4.2 Data Pipeline and Training Loop

Tool / model	Codex (OpenAI), GPT-5.6
Member / verifier	Tran Manh Thang (2353124)
Stage	September 2026, M1 Draft implementation

- **Purpose and scope:** Explain and review data loading, stratified splitting, training-only normalization, and epoch loops. Files: `src/data/` (dataset, split, statistics, transforms, pipeline, dataloader) and `src/training/` (train, validate, trainer, checkpoint).
- **Edits:** Adapted suggestions to the shared configuration and module interfaces; reused split indices for normalization and loading; integrated epoch loops with validation and checkpoints.
- **Verification:** Used PyTorch documentation (DataLoader, Normalize, CrossEntropyLoss), scikit-learn documentation (`train_test_split`), and `tests/test_data/` and `tests/test_training/`. Checked split overlap, coverage, and tensor shapes in the EDA notebook.

Representative prompt example

> "Review the Fashion-MNIST data pipeline and training loop. Check that normalization uses only the training subset, the fixed-seed stratified split has no overlapping indices, and epoch loss and accuracy are accumulated by sample count."

4.3 Linear, MLP, and CNN Models

Tool / model Codex (OpenAI), GPT-5.6
Member / verifier Hoang Kim Cuong (2352145)
Stage September 2026, M1 Draft implementation

- **Purpose and scope:** Explain activation and regularization choices and review Linear, MLP, and CNN implementations. Files: `src/models/linear.py`, `mlp.py`, `cnn.py`, and the matching YAML files in `config/`.
- **Edits:** Revised layer dimensions, activation and regularization placement, and forward passes to match the configurations and shared training interface; retained logits as outputs.
- **Verification:** Traced shapes manually and checked `tests/test_models/`. Compared implementations with PyTorch documentation (Linear, Conv2d, BatchNorm2d, CrossEntropyLoss); trained and evaluated all three models, saving outputs in `m1_draft_results/runs/`.

Representative prompt example

> "Review the Linear, MLP, and CNN classifiers. Explain why CrossEntropyLoss takes logits, check the CNN channel and spatial dimensions, and review the placement of dropout and batch normalization."

4.4 GRU and Transformer Models

Tool / model Claude (Anthropic), Claude Opus
Member / verifier Phan Tan Loc (2352712)
Stage September 2026, M2 Final preparation

- **Purpose and scope:** Explain image sequences, patch embedding, classification tokens, and positional encoding; review model implementations. Files: `src/models/rnn.py`, `transformer.py`, `config/rnn.yaml`, and `config/transformer.yaml`.
- **Edits:** Adapted suggestions to the row-wise GRU and patch-based Transformer; revised reshaping, patch tokens, positional embeddings, and classification heads to match configured dimensions and the shared image-to-logits interface.
- **Verification:** Checked shapes in `tests/test_models/test_gru.py` and `test_transformer.py`; consulted PyTorch documentation (GRU, LSTM, TransformerEncoderLayer, TransformerEncoder). GRU was the evaluated recurrent baseline.

Representative prompt example

"Explain how to represent a 28 by 28 image as a sequence for a GRU and check the timestep and feature dimensions. Review the Transformer's patch projection, classification token, positional embeddings, and encoder input shape."

4.5 Results Provenance and Responsibility

- **Results provenance:** Metrics come from executed training and evaluation runs. Learning curves visualize recorded epoch histories; confusion matrices and prediction examples use evaluated predictions and supplied labels. Saved experimental outputs support the numerical results.
- **Final responsibility:** The members named above verify the corresponding AI-assisted work. Group 09 retains responsibility for the submitted code, figures, results, and interpretations, and for explaining these contributions during assessment.

5 Reproducibility

The source repository provides the implementation and configurations. The baseline source tag is `a1-m2-baselines-2026-09-25`. Saved checkpoints are available in the M2 baseline release, in `a1-m2-baseline-runs-20260925-122457.zip`.

Model	Recorded baseline run ID
Linear	`20260924-124837-513213-m2-baseline`
MLP	`20260924-125349-995443-m2-baseline`
CNN	`20260924-125826-798568-m2-baseline`
GRU (`rnn`)	`20260924-130429-733285-m2-baseline`
Transformer	`20260924-130954-040349-m2-baseline`

Table 11: Saved runs underlying the reported baseline results.

Each run links `best_model.pt`, `training.json`, `evaluation.json`, and figures to its recorded configuration, data-split settings, and epoch logs. Installation, dataset preparation, training, and evaluation instructions are provided in the Assignment 1 README. Use the released code and matching configuration when evaluating a saved checkpoint: evaluation reads the configuration on disk. The source tag identifies the released reproduction code; it is not an automatically recorded training-time commit.

References

[1] Instructor's lecture slides. `08-MLP.pdf`.

[2] Instructor's lecture slides. `09-TrainingANN.pdf`.

[3] Instructor's lecture slides. `02-Datasets.pdf`.

[4] Instructor's lecture slides. `04-ClassificationSummary.pdf`.

[5] Instructor's lecture slides. `01-Conv2d-Overview.pdf`.

[6] Instructor's lecture slides. `RNN-Summary.pdf`.

[7] Instructor's lecture slides. `02-Attention-Transformer-Core.pdf`.

[8] Instructor's lecture slides. `06-Metrics.pdf`.

[9] Zalando Research. *Fashion-MNIST: Official Dataset Repository*. Dataset documentation and MIT license. `https://github.com/zalandoresearch/fashion-mnist`.